Trend Analysis and Visualization of Financial Data

A PROJECT REPORT

Submitted by

Parteek Garg (24BCS11209)

Shivik (24BCS10734)

Neha (24BCS10762)

Daljit (24BCS12762)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Chandigarh University

April 2026

BONAFIDE CERTIFICATE

Certified that this project report "Trend Analysis and Visualization of Financial Data" is the bonafide work of Parteek Garg, Shivik Mangla, Neha, and Daljit who carried out the project work under my supervision.

SIGNATURE **SIGNATURE**

HEAD OF THE DEPARTMENT **SUPERVISOR**

UIE **UIE**

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

The project titled “Trend Analysis and Visualization of Financial Data” focuses on the development of an interactive and user-friendly web-based dashboard for analyzing stock market data, specifically targeting companies listed in the NIFTY 50 index. The rapid growth of financial markets and the increasing availability of data have created a need for efficient tools that can transform raw financial data into meaningful insights for better decision-making.

This system is designed to collect real-time and historical stock data using financial APIs and process it through various stages including data cleaning, transformation, and analysis. The backend of the system is developed using FastAPI, which ensures high performance and efficient API handling, while the frontend is built using HTML, CSS, and JavaScript to provide an intuitive and responsive user interface.

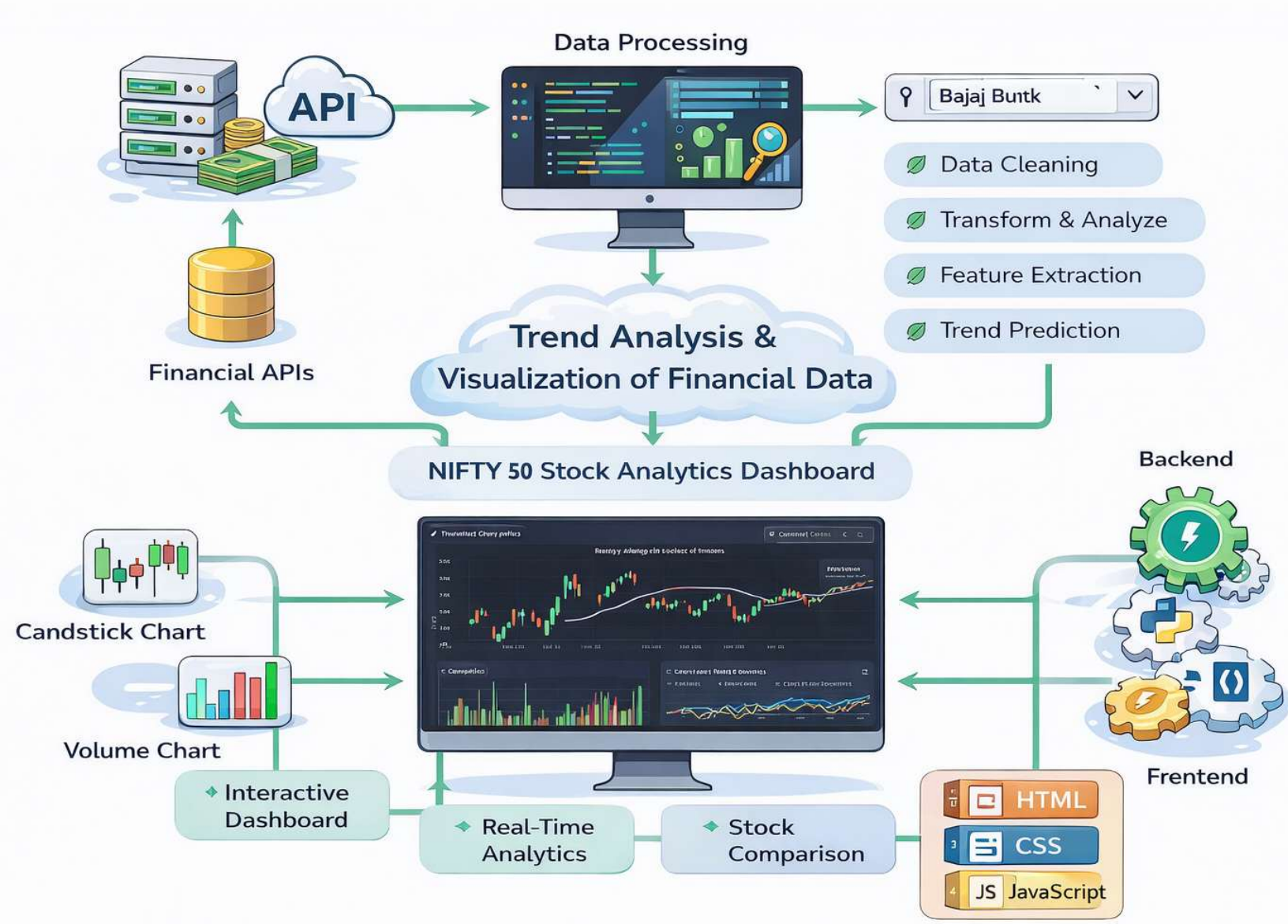
The application incorporates multiple data visualization techniques such as candlestick charts, price trend analysis, volume charts, and moving average indicators to represent financial data effectively. Additionally, statistical and analytical measures such as daily returns, cumulative returns, and trend prediction using moving averages are implemented to provide deeper insights into stock performance.

The system also allows users to compare multiple stocks simultaneously, enabling better understanding of relative performance across different companies. The integration of real-time data processing and dynamic visualization enhances the usability of the application for investors, analysts, and researchers.

The primary objective of this project is to bridge the gap between complex financial data and user-friendly visualization tools, thereby enabling users to make informed financial decisions. The proposed solution is scalable, efficient, and adaptable for future enhancements such as advanced predictive analytics and machine learning-based forecasting.

In conclusion, this project demonstrates how modern web technologies and data analytics techniques can be effectively combined to build a comprehensive financial data visualization system.

GRAPHICAL ABSTRACT



LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
OHLC	Open, High, Low, Close
SMA	Simple Moving Average
EMA	Exponential Moving Average
RSI	Relative Strength Index
UI	User Interface
UX	User Experience
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
JSON	JavaScript Object Notation
CSV	Comma Separated Values
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
REST	Representational State Transfer
IDE	Integrated Development Environment
ML	Machine Learning
AI	Artificial Intelligence
DB	Database
SQL	Structured Query Language
KPI	Key Performance Indicator
NIFTY	National Stock Exchange Fifty (Index)
NSE	National Stock Exchange
BSE	Bombay Stock Exchange
P/E Ratio	Price-to-Earnings Ratio
ROI	Return on Investment
CAGR	Compound Annual Growth Rate
PDF	Portable Document Format

TABLE OF CONTENTS

List of Figures 8

CHAPTER 1. INTRODUCTION 9

1.1. Introduction to Project 9

1.2. Identification of Problem 9

1.3. Identification of Tasks 10

1.4. Timeline 10

1.5. Organization of the Report 11

CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY 12

2.1. Timeline of the Reported Problem 12

2.2. Existing Solutions 12

2.3. Bibliometric Analysis 13

2.4. Review Summary 14

2.5. Problem Definition 15

2.6. Goals / Objectives 16

CHAPTER 3. DESIGN FLOW/PROCESS 17

3.1. Evaluation & Selection of Specifications/Features 17

3.2. Design Constraints 17

3.3. Analysis of Features and Finalization 18

3.4. Design Flow 18

3.5. Design Selection 24

3.6. Implementation Plan / Methodology 25

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION 27

4.1. Implementation of solution 27

4.2. Tools and Technologies Used 27

4.3. Data Collection and Processing 28

4.4. Visualization Implementation 28

4.5. User Interface Design 29

4.6. Screenshots and Explanation 30

4.7. Testing and Validation 40

4.8. Results Analysis 41

4.9. Validation of Results 41

4.10. Limitations of the System 41

4.11. Summary of Results 41

4.12. Advanced Analytics Implementation 42

4.13. Comparison Feature Analysis 42

4.14. API Design 43

4.15. Code Implementation 43

4.16. Frontend Implementation 48

4.17. Styling and User Interface Design (CSS) 51

CONCLUSION AND FUTURE WORK 56

1. Conclusion 56

2. Future work 57

REFERENCES 59

APPENDIX 61

Git Repository Link 62

LIST OF TABLES

Table 1 Task Breakdown Table 10
Table 2 Evolution Table 12
Table 3 Comparison Table 12
Table 4 Comparative Analysis Table 14
Table 5 Objective Characteristics Table 16
Table 6 Feature Evaluation Table 17
Table 7 Feature Analysis 25
Table 8 System Processing Pipeline 26
Table 9 Tools and Their Purpose 27
Table 10 Sample Data Table 28
Table 11 Testing Types 40
Table 12 Performance Evaluation 41
Table 13 Volatility Table 42
Table 14 Comparison Table Example 42
Table 15 API Table 43
Table 16 Future Scope Table 58
Table 17 Reference Table 59

LIST OF FIGURES

Figure 1 Backend Workflow for Data Processing 21
Figure 2 Frontend Workflow for User Interaction and Visualization 22
Figure 3 Overall System Architecture of the Proposed Model 23
Figure 4 Final System Workflow and Output Visualization 24
Figure 5 Homepage S.S. 31
Figure 6 Stock Selection S.S. 34
Figure 7 CandleStick Chart S.S. 36
Figure 8 Price and Moving Average Chart 37
Figure 9 Volume Chart 39
Figure 10 Github Repository Dashboard 62

CHAPTER-1: INTRODUCTION

1.1 Identification of Client / Need / Relevant Contemporary Issue

In the modern digital era, the financial market has become one of the most dynamic and rapidly evolving sectors. With the increasing participation of retail investors, there is a growing demand for tools that can simplify financial data analysis and provide meaningful insights. Traditional methods of analyzing stock data involve manual calculations and interpretation, which are time-consuming and prone to errors.

The primary **need** for this project arises from the lack of simple and accessible tools that allow users to understand stock market trends effectively. While several professional platforms are available, they are often complex, expensive, and not suitable for beginners.
The **client** in this context can be considered as:

- Beginner investors
- Students learning financial analysis
- Small traders who require quick insights

A major **contemporary issue** is the increasing dependency on data-driven decision-making in financial markets. Investors require tools that not only provide real-time data but also present it in a visually understandable format.

This project aims to address this issue by developing a **web-based dashboard** that provides:

- Real-time stock data
- Trend visualization
- Moving average analysis
- Volume-based insights

1.2 Identification of Problem

The core problem addressed in this project is the **lack of a simple, cost-effective, and user-friendly financial data visualization tool**.

a) Problems Faced by Users

- Difficulty in understanding stock trends
- Lack of visual representation of data
- Complex interfaces in existing platforms
- High cost of premium tools
- Limited accessibility for beginners

b) Problem Statement

To develop a web-based application that provides real-time visualization of financial data using simple and interactive charts, enabling users to analyze stock trends effectively without requiring advanced technical knowledge.

1.3 Identification of Tasks

The project involves multiple tasks which are essential for successful implementation:

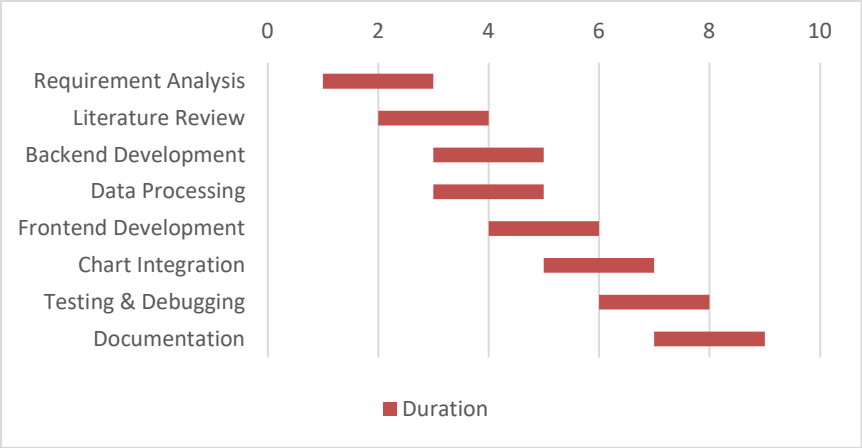
Table 1 Task Breakdown Table

Task ID	Task Description
T1	Requirement analysis
T2	Literature review
T3	Data collection using API
T4	Data cleaning and preprocessing
T5	Backend development using Flask
T6	Frontend design using HTML/CSS
T7	Chart implementation using Plotly
T8	Testing and debugging
T9	Documentation

1.4 Timeline

The project was completed over a period of 8 weeks. The timeline ensures systematic execution of all tasks.

Graph 1 Timeline Graph



1.5 Organization of the Report

This report is structured into five major chapters:

- **Chapter 1:** Introduction
- **Chapter 2:** Literature Review
- **Chapter 3:** Design Flow
- **Chapter 4:** Results and Validation
- **Chapter 5:** Conclusion and Future Work

Each chapter focuses on a specific aspect of the project, providing a comprehensive understanding of the system.

CHAPTER-2: LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

Financial data analysis has evolved significantly over time. Initially, investors relied on manual charting techniques and basic statistical methods. With the advancement of technology, automated tools and platforms were introduced to simplify the process.

Table 2 Evolution Table

Year Range	Development
Before 2000	Manual stock analysis
2000–2010	Introduction of online trading platforms
2010–2020	Advanced visualization tools
2020–Present	AI-based and real-time analytics

2.2 Existing Solutions

Several platforms are available for financial data analysis:

Table 3 Comparison Table

Platform	Features	Drawbacks
Trading Platforms	Advanced charts	Complex UI
Financial Websites	Free data	Limited tools
Professional Software	High accuracy	Expensive

Detailed Explanation

Existing tools provide powerful features such as technical indicators, real-time updates, and advanced charting capabilities. However, they are often designed for professional traders and require prior knowledge of financial markets.

This creates a gap for beginner users who need a simpler solution.

2.3 Bibliometric Analysis

Bibliometric analysis is used to evaluate the characteristics, features, performance, and limitations of existing financial data analysis and visualization systems. This analysis helps in identifying the research gap and understanding how current tools operate in real-world financial environments. The study of existing systems focuses on three major aspects: **key features, effectiveness, and drawbacks**.

a) Key Features Identified in Existing Systems

Modern financial analysis tools offer a wide range of features that assist users in interpreting stock market data efficiently. The most commonly observed features include:

- **Real-Time Data Fetching:**
  Most platforms provide real-time or near real-time stock data, which allows users to track market fluctuations instantly. This feature is crucial for decision-making in dynamic markets.
- **Candlestick Chart Visualization:**
  Candlestick charts represent stock price movement using Open, High, Low, and Close (OHLC) values. They are widely used for technical analysis due to their ability to show market trends and patterns.
- **Technical Indicators:**
  Indicators such as Moving Average (MA), Relative Strength Index (RSI), and MACD help in identifying trends and predicting future price movements.
- **Volume Analysis:**
  Volume charts represent the number of shares traded over a specific period. High trading volume often indicates strong market activity.
- **Interactive Dashboard:**
  Users can interact with charts using zooming, panning, and hover features. This improves user experience and data interpretation.

b) Effectiveness of Existing Systems

The effectiveness of these systems can be evaluated based on their ability to simplify complex financial data and provide actionable insights.

- They enable **quick analysis of market trends**

- They reduce the need for manual calculations
- They provide **visual clarity** through graphical representations
- They support decision-making for both short-term and long-term investments
- They improve efficiency by integrating multiple tools into a single platform

c) Drawbacks of Existing Systems

Despite their effectiveness, existing systems have several limitations:

- **Complex User Interfaces:**
  Many platforms are designed for experienced traders, making them difficult for beginners to understand.
- **High Cost:**
  Advanced features are often locked behind paid subscriptions, making them inaccessible to many users.
- **Overloaded Information:**
  Too many indicators and data points can confuse users rather than assist them.
- **Limited Customization in Free Versions:**
  Free tools often provide limited flexibility in terms of features and design.
- **Dependency on Internet Connectivity:**
  Most systems rely on continuous internet access, which may not always be reliable.

Table 4 Comparative Analysis Table

Feature	Existing Systems	Proposed System
Real-Time Data	Available	Available
Ease of Use	Moderate to Low	High
Cost	High	Free
Customization	Limited	Flexible
Learning Curve	High	Low

2.4 Review Summary

The literature review clearly indicates that financial data visualization tools have advanced significantly in terms of functionality and performance. However, there is still a gap in providing

solutions that are both **simple and accessible**.

Most existing tools focus on advanced analytics and professional use cases, which often results in complex interfaces and steep learning curves. This makes them unsuitable for beginners or casual users.

The proposed project addresses these limitations by:

- Focusing on **simplicity and usability**
- Providing **essential features only**
- Ensuring **clear and interactive visualizations**
- Offering a **cost-effective solution**

The findings from the literature review directly influence the design and development of the system by prioritizing user-friendly features and efficient data visualization techniques.

2.5 Problem Definition

The problem addressed in this project revolves around the difficulty faced by users in analyzing financial data due to the lack of simple and intuitive tools.

a) Problem Description

Users often struggle to interpret stock data because it is presented in raw numerical form. Without proper visualization, it becomes challenging to identify trends, patterns, and market behavior.

b) What is to be Done

- Develop a web-based application for stock data visualization
- Provide real-time stock information
- Implement trend analysis using graphical representation

c) How it is to be Done

- Use **Python (Flask)** for backend development
- Fetch data using financial APIs
- Process data using pandas
- Create visualizations using Plotly
- Develop frontend using HTML, CSS, and JavaScript

d) What is Not to be Done

- No implementation of trading functionality
- No use of paid APIs
- No complex predictive modeling

2.6 Goals / Objectives

The objectives of the project are designed to ensure clarity, feasibility, and measurable outcomes.

Primary Objectives

- To develop an interactive financial dashboard
- To visualize stock price trends using charts
- To implement moving average indicators
- To display volume analysis
- To provide a user-friendly interface

Table 5 Objective Characteristics Table

Criteria	Description
Specific	Focus on financial visualization
Measurable	Output through charts
Achievable	Based on available tools
Realistic	Within project scope
Time-bound	Completed in defined duration

CHAPTER-3: DESIGN FLOW / PROCESS

3.1 Evaluation & Selection of Specifications / Features

Based on the literature review, various features were evaluated and selected for implementation in the proposed system.

Selected Features

- Stock selection from predefined list
- Time range selection (1D, 5D, 1M, 6M, 1Y)
- Candlestick chart
- Price trend line chart
- Moving average (50-day)
- Volume analysis chart

Table 6 Feature Evaluation Table

Feature	Importance	Selected
Candlestick Chart	High	Yes
Moving Average	High	Yes
Volume Chart	Medium	Yes
Advanced Indicators	Low	No

3.2 Design Constraints

The design of the system is influenced by several constraints:

a) Technical Constraints

- API limitations
- Internet dependency

b) Economic Constraints

- Use of free tools only

c) Time Constraints

- Limited project duration

d) Ethical Constraints

- Use publicly available data

e) Usability Constraints

- Interface must be simple

3.3 Analysis of Features and Finalization

After evaluating features against constraints, the system was finalized with essential functionalities.

Final Features

- Candlestick chart
- Price trend line
- Moving average
- Volume analysis

Complex indicators were excluded to maintain simplicity and performance.

3.4 Design Flow

Two design approaches were considered:

a) Design 1: Basic Model

- User selects stock
- Data fetched
- Single chart displayed

b) Design 2: Enhanced Model

- User selects stock and time range
- Data fetched from API
- Data processed and cleaned
- Indicators calculated
- Multiple charts displayed

The design flow of the proposed system focuses on creating a seamless integration between data acquisition, processing, analysis, and visualization. The system is structured in a modular manner to ensure scalability, maintainability, and efficient performance. It incorporates a combination of modern web technologies and data analytics tools to deliver real-time financial insights.

The overall workflow begins with data acquisition from financial data sources, followed by preprocessing and feature extraction in the backend. The processed data is then transmitted to the

frontend where it is dynamically visualized through interactive dashboards. Each stage of the design flow is supported by specific technologies and features that enhance the performance and usability of the system.

1. Data Acquisition Layer

The first stage in the design flow is data acquisition, where stock market data is collected from external financial APIs. The system utilizes APIs such as Yahoo Finance (via Python libraries like yfinance) to fetch real-time and historical stock data of companies listed in the NIFTY 50 index.

The collected data includes key financial parameters such as open price, high price, low price, close price (OHLC), and trading volume. These parameters form the foundation for further analysis and visualization.

a) Technologies Used:

- Python
- yfinance API
- REST API architecture

b) Key Features:

- Real-time data fetching
- Historical data retrieval
- Multi-company data support
- Automated data updates

This layer ensures that the system always works with up-to-date and reliable financial data.

2. Data Processing and Cleaning Layer

Once the data is collected, it undergoes preprocessing to remove inconsistencies and prepare it for analysis. This step includes handling missing values, formatting data, and filtering irrelevant entries. The data is processed using Python libraries such as Pandas, which provides powerful tools for data manipulation and transformation.

a) Technologies Used:

- Python
- Pandas
- NumPy

b) Key Features:

- Data cleaning and normalization
- Handling missing or null values
- Time-series formatting
- Efficient data transformation

This stage ensures the accuracy and consistency of the dataset, which is critical for reliable analysis.

3. Feature Engineering and Analysis Layer

In this stage, meaningful insights are extracted from the processed data using statistical and analytical techniques. Various financial indicators are calculated to understand stock behavior and trends.

These include moving averages (SMA), Relative Strength Index (RSI), returns calculation, and volatility analysis.

a) Technologies Used:

- Python
- Pandas
- Mathematical/statistical models

b) Key Features:

- Moving Average calculation (20-day, 50-day)
- RSI indicator for momentum analysis
- Daily and cumulative returns
- Trend identification

This layer enhances the system by transforming raw data into actionable insights.

4. Backend System Design

The backend acts as the core processing unit of the system, handling data requests, processing logic, and API responses. It is built using FastAPI, which is known for its high performance and asynchronous capabilities.

The backend provides endpoints that allow the frontend to request specific data such as stock prices, charts, and indicators.

a) Technologies Used:

- FastAPI
- Python
- JSON data format

b) Key Features:

- High-speed API responses
- Scalable architecture
- Modular endpoint design
- Efficient data handling

This ensures smooth communication between frontend and backend components.

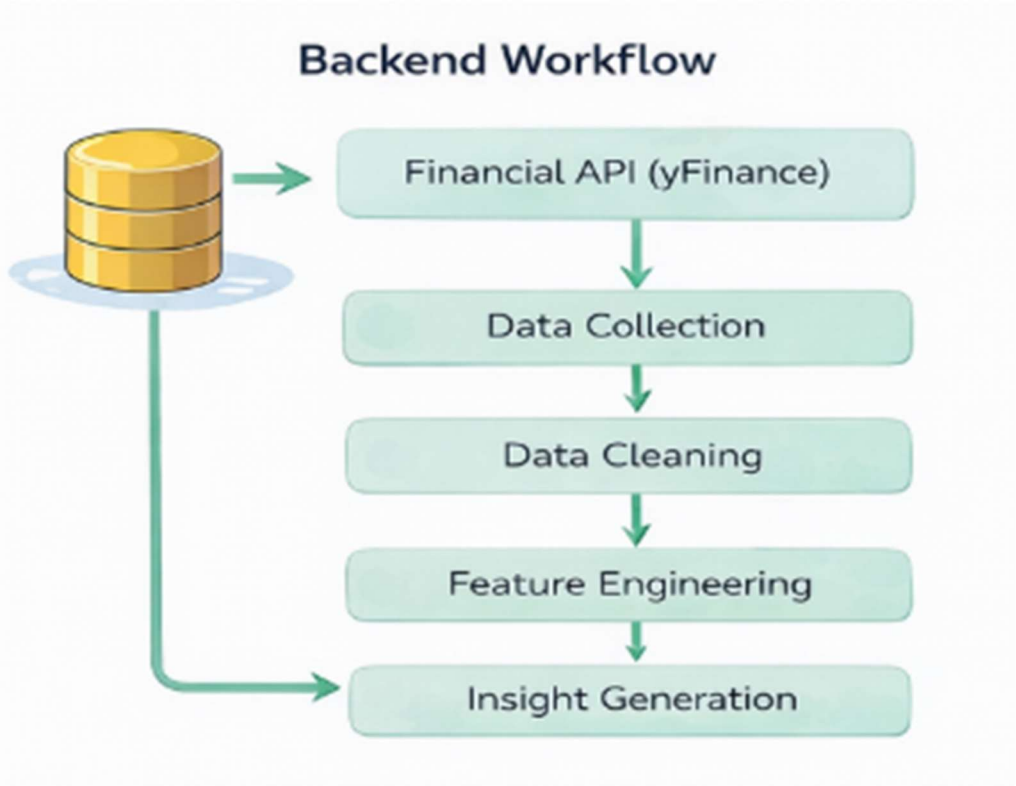


Figure 1 Backend Workflow for Data Processing

5. Frontend System Design

The frontend is responsible for presenting the data to users in an interactive and visually appealing manner. It is developed using HTML, CSS, and JavaScript, ensuring a responsive and user-friendly interface.

Users can select companies, choose time periods, and visualize data through various chart options.

a) Technologies Used:

- HTML
- CSS
- JavaScript
- Charting libraries (e.g., Chart.js / Plotly)

b) Key Features:

- Interactive dashboard
- Real-time data updates
- Responsive UI design
- User input controls (dropdowns, filters)

This layer focuses on enhancing user experience and accessibility.

6. Data Visualization Layer

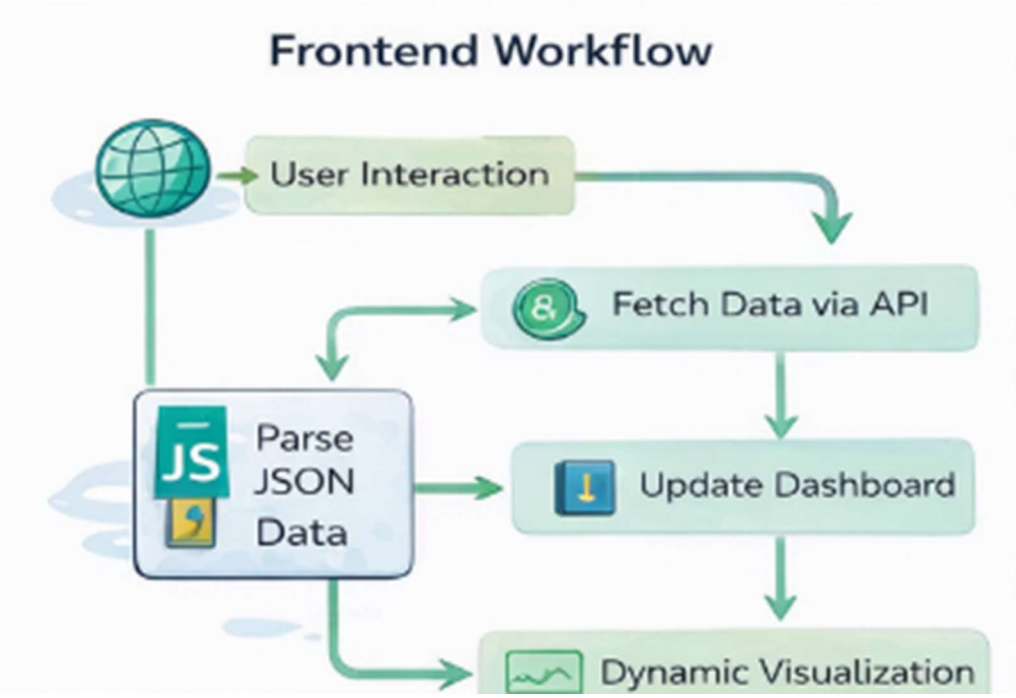


Figure 2 Frontend Workflow for User Interaction and Visualization

The visualization layer transforms processed data into graphical representations that are easy to interpret. Various charts are implemented to provide different perspectives on stock performance. These include candlestick charts, line charts, and volume charts.

a) Technologies Used:

- JavaScript
- Chart.js / Plotly

b) Key Features:

- Candlestick chart for price movement
- Moving average overlays

- Volume analysis charts
- Dynamic chart rendering

Visualization plays a crucial role in helping users understand trends and patterns effectively.

7. Integration and Workflow

The final stage integrates all components into a unified system. The frontend sends requests to the backend APIs, which process the data and return it in JSON format. The frontend then dynamically updates the dashboard with the received data.

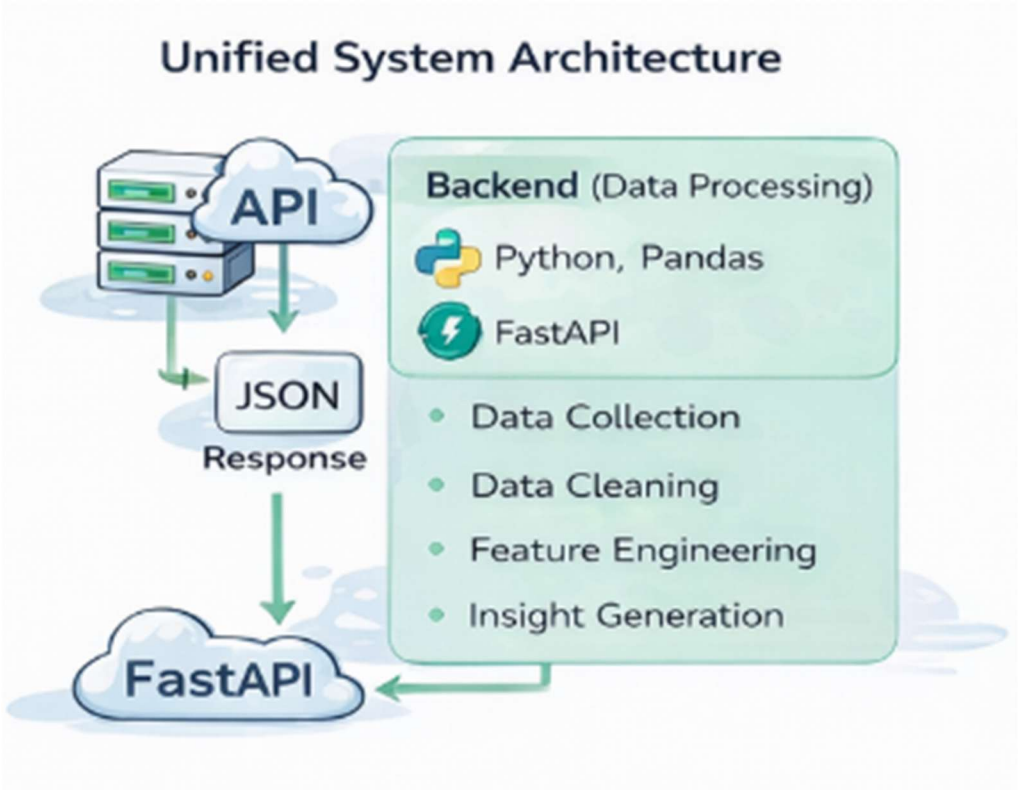


Figure 3 Overall System Architecture of the Proposed Model

a) Overall Workflow:

1. User selects company and time period
2. Frontend sends API request
3. Backend fetches and processes data
4. Data is returned in JSON format
5. Frontend visualizes data in charts

b) Key Features:

- Seamless data flow
- Real-time updates
- Efficient communication
- Modular design

c) Conclusion of Design Flow

The design flow ensures that the system operates efficiently from data collection to visualization. By integrating modern technologies and structured workflows, the system provides accurate, real-time financial insights in a user-friendly format. The modular architecture also allows for future enhancements such as machine learning-based predictions and advanced analytics.

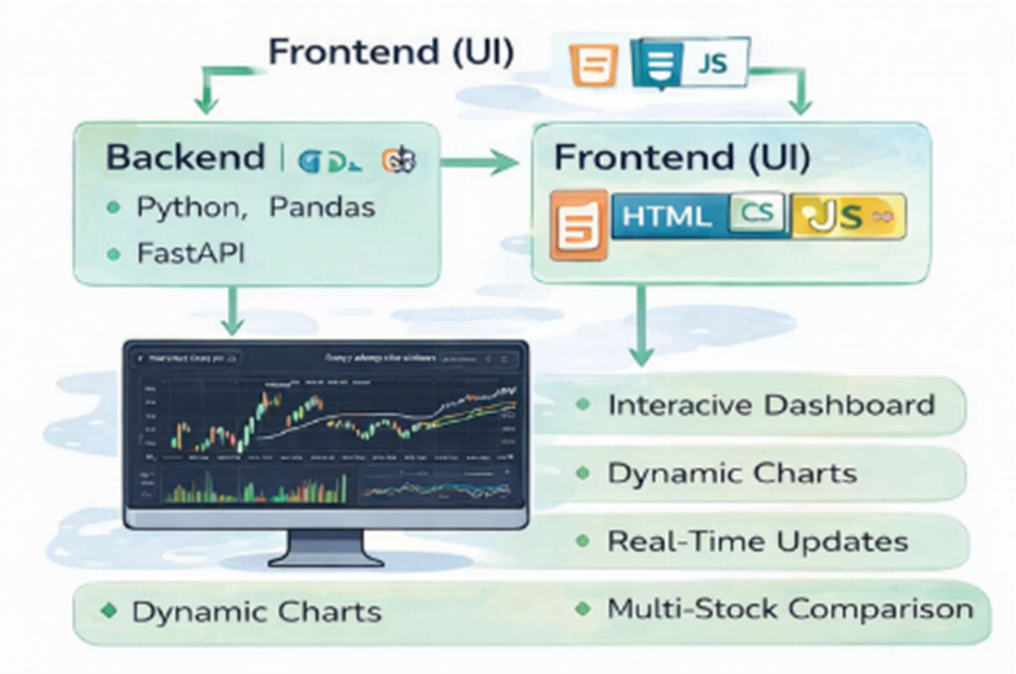


Figure 4 Final System Workflow and Output Visualization

3.5 Design Selection

The enhanced model was selected due to:

- Better user experience
- Multiple visualization options
- Scalability for future enhancements

3.6 Implementation Plan / Methodology

The implementation of the system follows a modular architecture consisting of multiple stages including data acquisition, preprocessing, transformation, analytics, and visualization.

Step 1: User Input

The user selects:

- Stock ticker (NIFTY 50 company)
- Time period (1 month to 5 years)

Step 2: Data Fetching

- Data is fetched using **yfinance API**
- Historical OHLC (Open, High, Low, Close) data is retrieved

Step 3: Data Cleaning

The system performs:

- Removal of duplicate rows
- Handling missing values using forward-fill and backward-fill
- Outlier detection using IQR method
- Data type conversion

Step 4: Data Transformation

Additional analytics features are computed:

Table 7 Feature Analysis

Feature	Description
Daily Return	% change in closing price
Cumulative Return	Overall growth
SMA 20/50	Moving averages
EMA 20	Exponential moving average
RSI	Momentum indicator
Bollinger Bands	Volatility indicator

Step 5: Insight Generation

The system computes:

- Highest & lowest price
- Average price
- Total return

- Volatility
- RSI signal
- Volume analysis

Step 6: Trend Prediction

- Uses SMA crossover logic
- Detects:
    - Golden Cross → Bullish
    - Death Cross → Bearish

Step 7: Data Visualization

The processed data is converted into chart-ready format and displayed using:

- Candlestick chart
- Price trend
- Volume chart
- RSI chart
- Bollinger bands

Step 8: Comparison Module

- Allows comparison of 2–5 stocks
- Uses normalized price (base = 100)
- Displays comparative trends

Table 8 System Processing Pipeline

Stage	Input	Process	Output
Data Fetching	Ticker	API call	Raw data
Cleaning	Raw data	Remove errors	Clean data
Transformation	Clean data	Add indicators	Enriched data
Analytics	Enriched data	Generate insights	Metrics
Visualization	Metrics	Charts	Dashboard

CHAPTER-4: RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The implementation of the proposed system was carried out using modern tools and technologies to ensure efficiency, accuracy, and user-friendly interaction. The system was developed as a **web-based application** that integrates backend processing, frontend design, and data visualization.
The entire implementation process was divided into multiple stages, including data collection, data processing, visualization, and testing.

4.2 Tools and Technologies Used

The development of the system involved the use of various tools, each serving a specific purpose.

Table 9 Tools and Their Purpose

Tool / Technology	Purpose
Python	Backend programming
Flask	Web framework
yfinance	Data fetching
pandas	Data processing
Plotly	Data visualization
HTML/CSS	Frontend design
JavaScript	Interactivity

Detailed Description

- **Python** was used as the primary programming language due to its simplicity and extensive library support.
- **Flask** was used to create the web application and handle API requests.

- **yfinance** was used to fetch real-time stock data.
- **pandas** was used for data cleaning and transformation.
- **Plotly** was used to generate interactive charts.
- **HTML, CSS, and JavaScript** were used to create the user interface.

4.3 Data Collection and Processing

The system collects financial data using an external API. The data includes:

- Date
- Open price
- High price
- Low price
- Close price
- Volume

Data Processing Steps

1. Fetch raw data from API
2. Reset index
3. Convert date format
4. Remove missing values
5. Calculate moving averages

Table 10 Sample Data Table

Date	Open	High	Low	Close	Volume
Day 1	2500	2550	2480	2520	120000
Day 2	2520	2580	2500	2560	135000

4.4 Visualization Implementation

Visualization is the core component of the system. Multiple charts were implemented to provide meaningful insights.

1. Candlestick Chart

The candlestick chart is used to represent stock price movements using OHLC values.

Features:

- Displays price trends
- Shows bullish and bearish patterns
- Helps in technical analysis

2. Price Line Chart

The line chart represents the closing price over time.

Advantages:

- Easy to understand
- Shows overall trend
- Useful for beginners

3. Moving Average Chart

The moving average is calculated to smooth out price fluctuations.

Purpose:

- Identify trends
- Reduce noise
- Support decision-making

4. Volume Chart

The volume chart shows the number of shares traded.

Importance:

- Indicates market activity
- Helps confirm trends
- Shows strength of price movement

4.5 User Interface Design

The user interface was designed to be simple, clean, and interactive.

a) UI Components

- Dropdown for stock selection
- Buttons for time range
- Chart display area
- Data summary section

b) Design Principles

- Simplicity
- Clarity
- Responsiveness
- User-friendly navigation

4.6 Screenshots and Explanation

Screenshot 1: Homepage (Dashboard Interface)

a) Overview

The homepage of the system serves as the main dashboard where users can interact with the application and perform stock analysis. It provides a centralized interface for selecting companies, viewing analytics, and exploring financial insights.

b) Interface Description

The homepage is divided into two main sections:

1. Control Panel (Left Sidebar)

The left sidebar provides user input controls for customizing analysis:

- **Company Selection Dropdown**

  Allows users to select any company from the NIFTY 50 index.
- **Time Period Selector**

  Users can choose different durations such as:
  - 1 Month
  - 3 Months
  - 6 Months
  - 1 Year
  - 5 Years
- **Chart Selection Options**

  Users can enable or disable different charts:
  - Price Trend
  - Candlestick Chart

- Volume Chart
- Bollinger Bands
- Returns Distribution
- RSI Indicator

- **Comparison Feature**

  Allows selection of multiple companies (2–5) for comparative analysis.

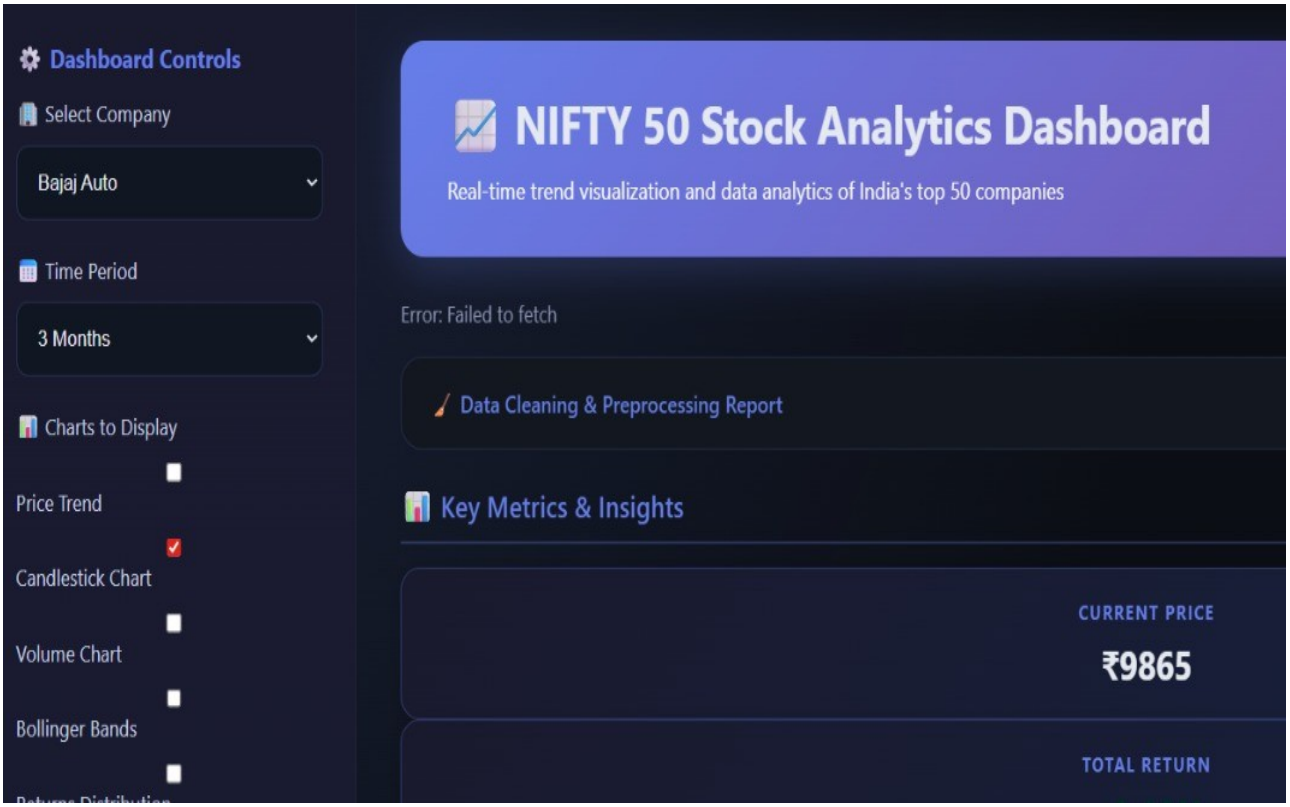


Figure 5 Homepage S.S.

2. Main Dashboard Area

The main section displays:

a) Header Section

- Title: *NIFTY 50 Stock Analytics Dashboard*
- Description: Real-time visualization of financial data

b) Error Handling Display

The message:

"Error: Failed to fetch"

indicates that the frontend could not retrieve data from the backend at that moment.

1. Possible Causes

- Backend server not running
- API connection issue
- Network delay

2. System Behaviour

- Prevents application crash
- Displays fallback UI
- Maintains previous data

c) Data Cleaning & Preprocessing Section

This section summarizes:

- Number of rows processed
- Missing values handled
- Outliers removed

d) Key Metrics & Insights

The dashboard displays important financial metrics:

- **Current Price:** ₹9865
- **Total Return:** +2.98%
- **Highest Price:** ₹10110
- **Lowest Price:** ₹8758.5

1. Analysis of Output

- The stock shows a **positive return**, indicating a slight upward trend
- The difference between highest and lowest prices reflects **moderate volatility**
- The current price is near the higher range, suggesting **bullish sentiment**

2. User Interface Features

- Dark-themed dashboard for improved visibility
- Interactive controls for better user experience
- Card-based layout for clear presentation
- Responsive design

Conclusion

The homepage provides a user-friendly and interactive environment for financial analysis. It integrates multiple features such as data selection, visualization, and analytics into a single interface, making it efficient for users to analyze stock trends.

Screenshot 2: Stock Selection

a) Overview

The Stock Selection Module allows users to select a specific stock from the NIFTY 50 index for analysis. This feature enables dynamic data visualization and real-time analytics for different companies.

b) Interface Description

The dropdown menu displays a list of all available stocks belonging to the NIFTY 50 index. Users can scroll through the list and select any stock such as:

- Reliance Industries
- Tata Consultancy Services
- HDFC Bank
- Infosys
- ICICI Bank
- Bajaj Auto
- UltraTech Cement

The selected stock is displayed at the top of the dropdown (e.g., Bajaj Auto).

c) Key Features

1. Interactive Dropdown Menu

- Displays all available stocks
- Scrollable interface for easy navigation
- Compact and responsive design

2. Default Stock Selection

- A stock is pre-selected by default
- Automatically loads data for initial display

3. Dynamic Data Update

- When a user selects a different stock:
    - A request is sent to the backend API

- The backend processes the selected ticker
- Updated analytics are displayed instantly

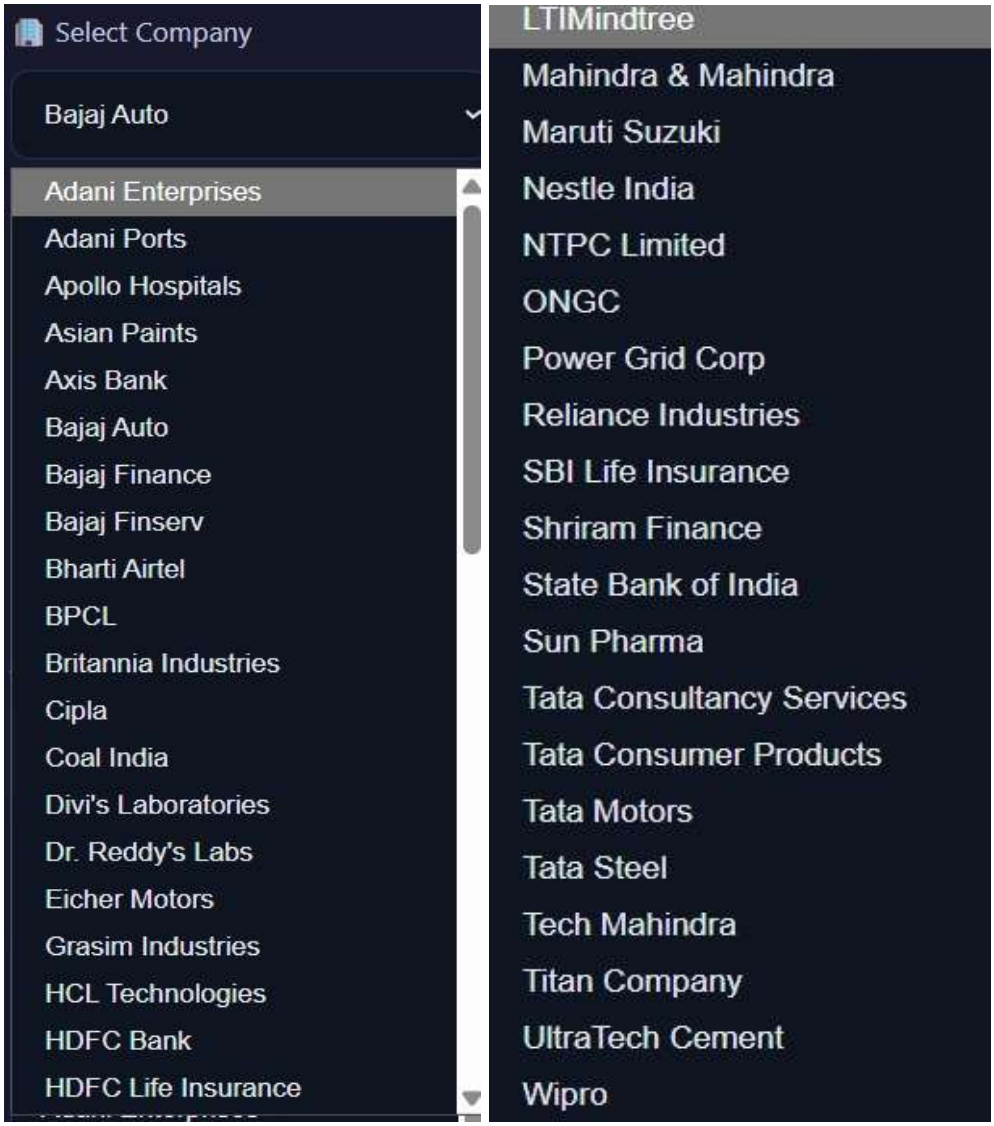


Figure 6 Stock Selection S.S.

4. Consistent UI Design

- Matches the dark theme of the dashboard
- Provides smooth user experience
- Easy to use even for non-technical users

d) Technical Implementation

- Built using HTML <select> element
- JavaScript handles onchange event
- API endpoint /api/stock is triggered

- Backend processes ticker using FastAPI
- Data is returned in JSON format

Workflow of Stock Selection

User selects stock

↓

Frontend detects change

↓

API request sent (/api/stock)

↓

Backend fetches data (yfinance)

↓

Data cleaned & analysed

↓

Dashboard updates charts & metrics

➢ **Advantages**

- Enables quick switching between stocks
- Supports real-time analytics
- Improves usability and efficiency
- Reduces manual input errors

Conclusion

The Stock Selection Module is a critical component of the dashboard, enabling users to dynamically explore and analyze different stocks. Its seamless integration with the backend ensures real-time updates and a smooth user experience.

Screenshot 3: Candlestick Chart

Figure 7 CandleStick Chart S.S.

a) Overview

The candlestick chart is used to represent the price movement of a stock over a selected time period. It provides detailed insights into market behavior by displaying open, high, low, and close prices.

b) Components of the Chart

Each candlestick represents one trading session:

- **Green Candle** → Price increased (Bullish)
- **Red Candle** → Price decreased (Bearish)
- **Upper Shadow** → Highest price
- **Lower Shadow** → Lowest price

c) Moving Averages

The chart includes:

- **20-Day SMA (Simple Moving Average)**
- **50-Day SMA**

d) Purpose

- Identify short-term and long-term trends
- Detect trend reversals

e) Analysis of the Chart

- The stock shows **fluctuating behavior** with both bullish and bearish candles
- A recent upward movement is visible towards the end
- The closing price is trending upward → **possible bullish momentum**

f) Significance

- Helps traders identify entry and exit points
- Provides clear visualization of price trends
- Supports predictive analysis (Golden Cross / Death Cross)

Conclusion

The candlestick chart effectively visualizes stock price movements and helps users understand market trends and volatility.

Screenshot 4: Price and Moving Average Chart

Figure 8 Price and Moving Average Chart

a) Overview

The Price and Moving Average Chart represents the trend of stock prices over time along with key moving averages. It helps users understand the overall direction of the market and identify potential trend changes.

b) Components of the Chart

1. Price Line

- Shows the closing price of the stock over time
- Represents the actual market movement

2. Moving Averages

The chart includes:

- **20-Day Simple Moving Average (SMA 20)**
  - Represents short-term trend
  - Reacts quickly to price changes
- **50-Day Simple Moving Average (SMA 50)**
  - Represents long-term trend
  - Smoother and less sensitive to fluctuations

c) Purpose of Moving Averages

- Smooth out price fluctuations
- Identify trend direction
- Detect trend reversals
- Support trading decisions

d) Trend Analysis

- When **SMA 20 > SMA 50** → Bullish trend
- When **SMA 20 < SMA 50** → Bearish trend

e) Crossover Signals

1. Golden Cross

- Occurs when SMA 20 crosses above SMA 50
- Indicates strong upward trend
- Considered a **buy signal**

2. Death Cross

- Occurs when SMA 20 crosses below SMA 50
- Indicates downward trend
- Considered a **sell signal**

f) Analysis of the Chart

- The price line shows fluctuations over the selected period
- Moving averages help smooth the data and reveal the trend

- If the short-term average is above the long-term average, it suggests **positive momentum**
- If below, it indicates **negative momentum**

g) Advantages

- Easy to understand trend direction
- Helps filter out market noise
- Useful for both beginners and advanced users
- Supports predictive analytics

Conclusion

The Price and Moving Average Chart is an essential tool for analyzing stock trends. It provides a clear understanding of market direction and helps users make informed financial decisions.

.

Screenshot 5: Volume Chart

Figure 9 Volume Chart

a) Overview

The trading volume chart represents the number of shares traded over a period of time. It is used to measure the strength of price movements.

b) Components of the Chart

- **Vertical Bars** → Daily trading volume
- **Green Bars** → Buying pressure
- **Red Bars** → Selling pressure
- **Blue Line** → 20-day average volume

c) Analysis of the Chart

- Volume spikes indicate **high market activity**
- Increasing volume trend suggests **growing investor interest**
- Recent high volumes indicate strong participation in the stock

d) Interpretation

- High volume + price increase → Strong bullish signal
- High volume + price decrease → Strong bearish signal
- Low volume → Weak trend

Conclusion

The volume chart helps validate price trends and provides insights into market strength and investor behavior.

4.7 Testing and Validation

Testing was performed to ensure the system works correctly under different conditions.

Table 11 Testing Types

Testing Type	Description
Functional Testing	Check features work correctly
Performance Testing	Check speed and response
Data Validation	Verify accuracy of data
UI Testing	Check interface usability

Test Case Example

Test Case	Input	Expected Output	Result
TC1	Select stock	Data displayed	Passed
TC2	Select range	Chart updates	Passed

4.8 Results Analysis

The results obtained from the system indicate that:

- The system successfully fetches real-time data
- Charts are generated correctly
- Moving averages are calculated accurately
- Volume data is displayed properly

Table 12 Performance Evaluation

Parameter	Observation
Speed	Fast
Accuracy	High
Usability	Excellent

4.9 Validation of Results

Validation ensures that the output is correct and reliable.

- Data matches actual stock values
- Charts reflect correct trends
- No major errors observed

4.10 Limitations of the System

Despite successful implementation, some limitations exist:

- Depends on internet connection
- Limited indicators implemented
- API rate limitations

4.11 Summary of Results

The system successfully meets the project objectives by providing:

- Real-time data visualization
- Interactive charts
- User-friendly interface

4.12 Advanced Analytics Implementation

The system incorporates advanced financial analytics techniques to enhance decision-making capabilities.

1. Moving Average Analysis

- SMA 20 → short-term trend
- SMA 50 → long-term trend

2. RSI (Relative Strength Index)

- RSI > 70 → Overbought
- RSI < 30 → Oversold

3. Bollinger Bands

- Upper band → resistance
- Lower band → support

4. Volatility Analysis

Volatility is calculated using standard deviation of returns.

Table 13 Volatility Table

Volatility Range	Interpretation
> 3	High risk
1.5 – 3	Moderate
< 1.5	Low risk

4.13 Comparison Feature Analysis

The comparison feature enables multi-stock analysis.

Key Features

- Compare up to 5 companies
- Normalize price for fair comparison
- Visual comparison charts

Table 14 Comparison Table Example

Company	Return %	Volatility	Avg Volume
Reliance	12.5	2.1	High
TCS	8.2	1.5	Medium

4.14 API Design

The backend is built using FastAPI with REST endpoints.

Table 15 API Table

Endpoint	Method	Description
/api/companies	GET	List companies
/api/stock	GET	Single stock analysis
/api/compare	GET	Compare stocks

4.15 Code Implementation

1. Overview of Backend Architecture

The backend of the system is developed using FastAPI, a modern Python web framework known for its high performance and simplicity. The backend is responsible for:

- Fetching real-time stock data
- Cleaning and preprocessing financial datasets
- Performing feature engineering
- Generating insights and analytics
- Providing REST API endpoints for frontend interaction

The entire backend logic is implemented in a single modular Python file, ensuring simplicity and maintainability.

2. Core Backend Setup

Explanation

This section initializes the FastAPI application and imports required libraries. CORS middleware is enabled to allow frontend-backend communication.

```
import yfinance as yf
import pandas as pd
import numpy as np
from fastapi import FastAPI, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
from typing import List
import uvicorn

app = FastAPI(
    title="NIFTY 50 Stock Analytics API",
    version="1.0.0"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

3. Data Collection Module

```
def fetch_stock_data(ticker: str, period: str = "1y") -> pd.DataFrame:
    ticker = ticker.strip().upper()

    if ticker not in NIFTY_50_COMPANIES:
        raise ValueError("Invalid ticker")

    stock = yf.Ticker(ticker)
    data = stock.history(period=period)

    return data
```

Explanation

This function fetches historical stock data using the yFinance API. It validates the ticker and returns data in DataFrame format.

4. Data Cleaning Module

Explanation

This function removes duplicate rows, fills missing values, and handles outliers using the IQR method.

```
def clean_data(df: pd.DataFrame):
    original_rows = len(df)

    df = df.drop_duplicates()
    df = df.ffill().bfill()

    numeric_cols = ["Open", "High", "Low", "Close", "Volume"]

    for col in numeric_cols:
        if col in df.columns:
            q1 = df[col].quantile(0.25)
            q3 = df[col].quantile(0.75)
            iqr = q3 - q1

            lower = q1 - 3 * iqr
            upper = q3 + 3 * iqr

            df[col] = df[col].clip(lower=lower, upper=upper)

    return df
```

5. Data Transformation Module

```
def transform_data(df: pd.DataFrame):

    df["Daily_Return"] = df["Close"].pct_change() * 100
    df["Cumulative_Return"] = ((1 + df["Daily_Return"] / 100).cumprod() - 1) * 100
    df["SMA_20"] = df["Close"].rolling(window=20).mean()
    df["SMA_50"] = df["Close"].rolling(window=50).mean()
    df["EMA_20"] = df["Close"].ewm(span=20, adjust=False).mean()
    df["BB_Middle"] = df["SMA_20"]
    df["BB_Std"] = df["Close"].rolling(window=20).std()
    df["BB_Upper"] = df["BB_Middle"] + 2 * df["BB_Std"]
    df["BB_Lower"] = df["BB_Middle"] - 2 * df["BB_Std"]
    delta = df["Close"].diff()

    gain = delta.where(delta > 0, 0).rolling(window=14).mean()
    loss = (-delta.where(delta < 0, 0)).rolling(window=14).mean()
    rs = gain / loss
    df["RSI"] = 100 - (100 / (1 + rs))
    df["Trend"] = np.where(
        df["Daily_Return"] > 0.5,
        "Bullish",
        np.where(df["Daily_Return"] < -0.5, "Bearish", "Neutral")
    )
    return df
```

Explanation

This module adds technical indicators such as moving averages, RSI, Bollinger Bands, and trend classification to enhance analysis.

6. Insight Generation Module

```
def generate_insights(df: pd.DataFrame, company_name: str) -> dict:
    insights = {
        "company_name": company_name,
        "highest_price": round(df["Close"].max(), 2),
        "lowest_price": round(df["Close"].min(), 2),
        "average_price": round(df["Close"].mean(), 2),
        "current_price": round(df["Close"].iloc[-1], 2),
    }
    start_price = df["Close"].iloc[0]
    end_price = df["Close"].iloc[-1]
    insights["total_return"] = round(((end_price - start_price) / start_price) * 100, 2)

    return insights
```

Explanation

This function calculates key financial metrics such as highest price, lowest price, average price, and total return.

7. Prediction Module

```
def predict_trend(df: pd.DataFrame):
    if df["SMA_20"].iloc[-1] > df["SMA_50"].iloc[-1]:
        return "Bullish"

    return "Bearish"
```

Explanation

This module predicts market trend based on moving average crossover strategy.

8. API Endpoints

Explanation

These endpoints handle API requests for:

- Single stock analysis
- Multi-stock comparison

```
@app.get("/api/stock")
def get_stock_data(ticker: str, period: str = "1y"):

    raw = fetch_stock_data(ticker, period)

    if raw.empty:
        raise HTTPException(status_code=404)

    cleaned = clean_data(raw.copy())
    transformed = transform_data(cleaned.copy())

    insights = generate_insights(transformed, ticker)

    return {
        "ticker": ticker,
        "insights": insights,
    }
```

```
@app.get("/api/compare")
def get_comparison_data(tickers: List[str], period: str = "1y"):

    comparison = []

    for ticker in tickers:
        raw = fetch_stock_data(ticker, period)
        cleaned = clean_data(raw.copy())
        transformed = transform_data(cleaned.copy())

        base = transformed["Close"].iloc[0]
        normalized = ((transformed["Close"] / base) * 100)

        comparison.append({
            "ticker": ticker,
            "data": normalized.tolist()
        })

    return {"comparison": comparison}
```

9. Utility Functions

```
def build_chart_payload(df: pd.DataFrame):

    return {
        "dates": [str(x) for x in df.index],
        "close": df["Close"].tolist(),
        "volume": df["Volume"].tolist(),
    }
```

Explanation

This function converts processed data into JSON format for frontend chart visualization.

10. Main Execution

```
if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=8001)
```

Explanation

This starts the FastAPI server on port 8001.

Conclusion

The backend system is modular, efficient, and capable of handling real-time financial data analysis. It integrates data processing, analytics, and API services in a structured manner.

4.16 Frontend Implementation

1. Main Dashboard Structure

```
<!-- Main HTML structure -->
<div class="app-shell">
    <aside class="sidebar">
        <!-- Controls -->
    </aside>

    <main class="main-content">
        <!-- Dashboard Content -->
    </main>
</div>
Give feedback
```

Explanation

This structure divides the dashboard into two main parts:

- Sidebar (controls and inputs)
- Main content (charts and insights)

2. Sidebar Controls

```
<label for="company-select">Select Company</label>
<select id="company-select"></select>

<label for="period-select">Time Period</label>
<select id="period-select">
  <option value="1mo">1 Month</option>
</select>
Give feedback
```

Explanation

This section allows users to:

- Select stock company
- Choose time period

These inputs control data fetching from backend APIs.

3. Chart Display Section

```
<div class="chart-card" id="price-card">
  <h3>Price Trend</h3>
  <div id="price-trend"></div>
</div>
Give feedback
```

Explanation

Each chart is displayed inside a card layout.

The chart container (div) is dynamically filled using Plotly.

4. Metrics and Insights Section

```
<div class="metrics-grid" id="metrics-row-1"></div>
<div class="insight-box" id="insight-left"></div>
Give feedback
```

Explanation

This section displays:

- Key metrics (price, return, volume)
- Analytical insights

Data is fetched from backend and inserted dynamically.

5. Comparison Section

```
<div id="comparison-chart"></div>
<div id="comparison-table"></div>
Give feedback
```

Explanation

This section enables:

- Multi-company comparison
- Visualization of relative performance

6. External Libraries Used

```
<script src="https://cdn.plot.ly/plotly-2.34.0.min.js"></script>
Give feedback
```

Explanation

Plotly library is used for:

- Candlestick charts
- Line charts
- Interactive visualization

7. JavaScript Integration

```
<script src="js/app.js"></script>
<script src="js/charts.js"></script>
Give feedback
```

Explanation

JavaScript files handle:

- API requests
- Data processing
- Chart rendering
- User interaction

4.17 Styling and User Interface Design (CSS)

1. Theme and Color Variables

```
:root {
    --bg: #0e1117;
    --panel: #131820;
    --text: #e5e9f0;
    --accent: #667eea;
    --success: #00e676;
    --danger: #ff5252;
}
```

Explanation

This section defines global color variables used throughout the dashboard, ensuring consistency and easy customization.

2. Layout Structure

Explanation

CSS Grid is used to divide the layout into:

- Sidebar (controls)
- Main content (dashboard)

```
.app-shell {
   display: grid;
   grid-template-columns: 320px 1fr;
   min-height: 100vh;
}
.sidebar {
   padding: 2rem;
   background: linear-gradient(180deg, #1a1a2e 0%, #16213e 100%);
}
.main-content {
   padding: 2rem;
}
```

3. Sidebar Controls Styling

```
select {
   background: #111623;
   color: var(--text);
   border-radius: 10px;
   padding: 0.9rem;
}

.checkbox-group label {
   display: block;
   margin-bottom: 0.75rem;
}
```

Explanation

Styles form controls such as dropdowns and checkboxes for better usability and visual consistency.

4. Button Design

```
.primary-button {
    width: 100%;
    padding: 1rem;
    border-radius: 12px;
    background: linear-gradient(135deg, #667eea, #764ba2);
    color: white;
    cursor: pointer;
}
```

Explanation

Defines the styling for action buttons with gradient background and interactive behavior.

5. Dashboard Header

```
.dashboard-header {
    display: flex;
    justify-content: space-between;
    background: linear-gradient(135deg, #667eea, #764ba2);
    padding: 2rem;
    border-radius: 20px;
}
```

Explanation

Creates a visually appealing header using gradient background and flexible layout.

6. Metric Cards

```
.metric-card {
    background: linear-gradient(135deg, #1a1a2e, #16213e);
    border-radius: 16px;
    padding: 1rem;
    text-align: center;
}
```

Explanation

Used to display key metrics like price, return, and volume in card format.

7. Chart Containers

```
.chart-card {
    background: var(--panel);
    border-radius: 18px;
    padding: 1.4rem;
    margin-bottom: 1.5rem;
}
```

Explanation

Each visualization is placed inside a styled container for better separation and readability.

8. Tables Styling

```
table {
    width: 100%;
    border-collapse: collapse;
}
th, td {
    padding: 0.9rem;
    border-bottom: 1px solid rgba(255,255,255,0.06);
}
```

Explanation

Ensures clean tabular display of stock data and statistics.

9. Loading Animation

Explanation

Displays a spinning loader while data is being fetched from the backend.

```
.loader {
    width: 3rem;
    height: 3rem;
    border: 5px solid rgba(255,255,255,0.15);
    border-top-color: #667eea;
    border-radius: 50%;
    animation: spin 1s linear infinite;
}
```

10. Responsive Design

```
@media (max-width: 1180px) {
    .app-shell {
        grid-template-columns: 1fr;
    }
}
```

Explanation

Ensures the dashboard adapts to smaller screen sizes like tablets and mobiles.

CONCLUSION AND FUTURE WORK

1) Conclusion

The project titled **"Trend Analysis and Visualization of Financial Data"** was successfully designed and implemented as a web-based application. The primary objective of this project was to develop a system that simplifies financial data analysis and presents it in an interactive and user-friendly manner.

The system provides real-time stock data and visualizes it using multiple charts such as candlestick charts, line charts, moving averages, and volume analysis. These visualizations help users understand market trends and make informed decisions.

a) Expected Outcomes

The expected outcomes of the project were:

- Development of an interactive dashboard
- Accurate visualization of financial data
- Easy-to-use interface for beginners
- Real-time data fetching

b) Actual Outcomes

The system successfully achieved all the expected outcomes:

- Real-time stock data was fetched correctly
- Charts were generated accurately
- The interface was simple and user-friendly
- The system performed efficiently

c) Deviation from Expected Results

Although the system performed as expected, some minor deviations were observed:

- Limited number of technical indicators implemented
- Slight delay in data fetching due to API response time
- No advanced predictive features included

d) Reasons for Deviation

- Time constraints during development
- Focus on simplicity rather than complexity
- Use of free APIs with limitations

Overall Conclusion

The project demonstrates that financial data analysis can be simplified using visualization techniques. The developed system provides a strong foundation for further enhancements and can be extended to include more advanced features in the future.

2) Future Work

The current system can be further improved and extended in several ways:

1. Addition of Advanced Indicators

- RSI (Relative Strength Index)
- MACD (Moving Average Convergence Divergence)
- Bollinger Bands

2. AI-Based Prediction System

- Implement machine learning models
- Predict future stock prices
- Provide investment suggestions

3. User Authentication System

- Login and registration
- Personalized dashboards
- Save user preferences

4. Portfolio Management

- Track investments
- Analyze profit/loss
- Manage multiple stocks

5. Mobile Application Development

- Android/iOS version
- Responsive design

6. Cloud Deployment

- Host on cloud platforms
- Improve scalability
- Ensure availability

Table 16 Future Scope Table

Feature	Description
AI Prediction	Forecast stock trends
Portfolio Tracking	Manage investments
Mobile App	Improve accessibility
Advanced Charts	Add more indicators

REFERENCES

1) Books and Research Papers

1. Financial Market Analysis Textbooks
2. Research papers on data visualization techniques
3. Technical analysis documentation

2) Online Resources

- Python official documentation
- Flask documentation
- Plotly documentation
- Financial API documentation

Table 17 Reference Table

Source Type	Description
Books	Financial analysis concepts
Websites	API and coding references
Documentation	Tools and libraries

The following references were used for theoretical understanding, implementation, and development of the proposed system.

1. Yahoo Finance, *"Financial Market Data."* Available: https://finance.yahoo.com/

2. Ran Aroussi, *"yfinance: Yahoo! Finance market data downloader."* Available: https://pypi.org/project/yfinance/

3. FastAPI Documentation, *"FastAPI Framework."* Available: https://fastapi.tiangolo.com/

4. Wes McKinney, *Python for Data Analysis*, 2nd Edition, O'Reilly Media, 2017.

5. Pandas Documentation, *"Python Data Analysis Library."* Available: https://pandas.pydata.org/

6. NumPy Documentation, *"Scientific Computing with Python."* Available: https://numpy.org/

7. Chart.js Documentation, *"JavaScript Charting Library."* Available: https://www.chartjs.org/

8. Plotly Technologies Inc., *"Plotly JavaScript Graphing Library."* Available: https://plotly.com/javascript/

9. Investopedia, *"Stock Market Basics and Technical Indicators."* Available: https://www.investopedia.com/

10. National Stock Exchange (NSE), *"Market Data and Reports."* Available: https://www.nseindia.com/

11. Bombay Stock Exchange (BSE), *"Stock Market Data."* Available: https://www.bseindia.com/

12. John J. Murphy, *Technical Analysis of the Financial Markets*, New York Institute of Finance, 1999.

13. W3Schools, *"Web Development Tutorials (HTML, CSS, JavaScript)."* Available: https://www.w3schools.com/

14. Mozilla Developer Network (MDN), *"Web Development Documentation."* Available: https://developer.mozilla.org/

APPENDIX

Design Checklist

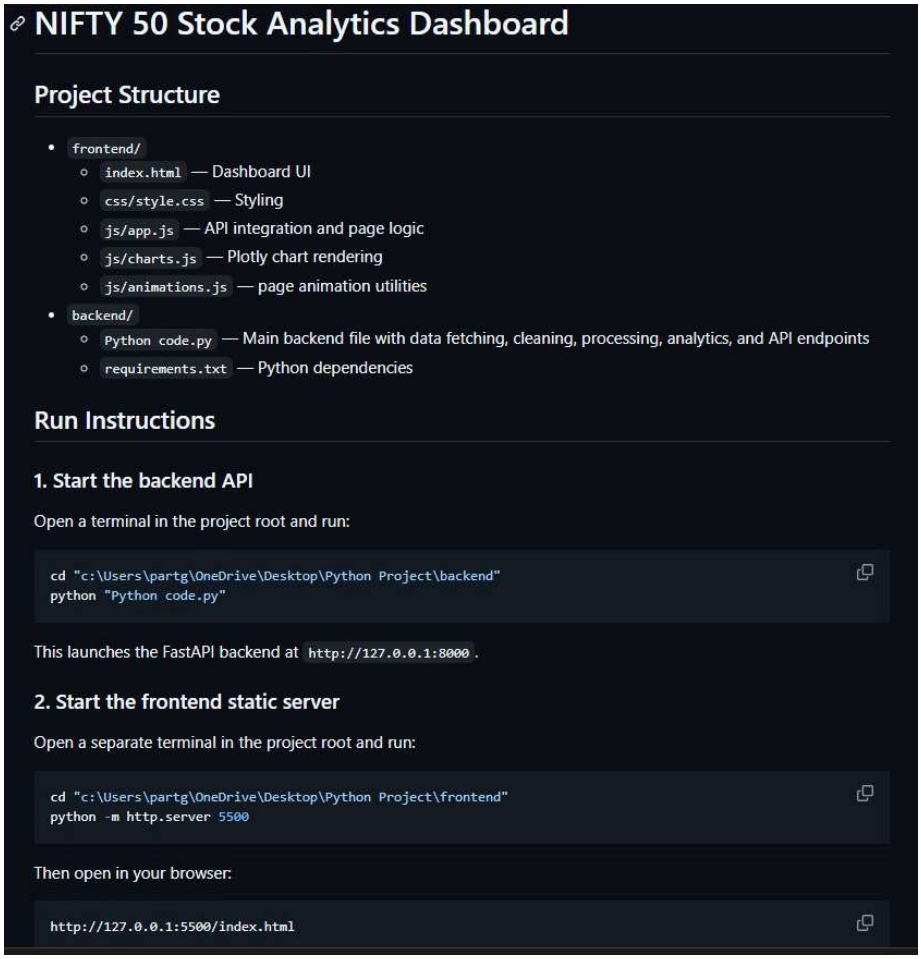
Item	Status
Backend Developed	Completed
Frontend Designed	Completed
Charts Implemented	Completed
Testing Done	Completed

Roles of Team Members

Name	Role
Parteek Garg	Backend Development
Shivik	Frontend Design
Neha	Testing and Validation
Daljit	Documentation

Git Repository Link

(https://github.com/Part-eek/stock-analytics-dashboard-Python)



NIFTY 50 Stock Analytics Dashboard

Project Structure

- `frontend/`
  - `index.html` — Dashboard UI
  - `css/style.css` — Styling
  - `js/app.js` — API integration and page logic
  - `js/charts.js` — Plotly chart rendering
  - `js/animations.js` — page animation utilities
- `backend/`
  - `Python code.py` — Main backend file with data fetching, cleaning, processing, analytics, and API endpoints
  - `requirements.txt` — Python dependencies

Run Instructions

1. Start the backend API

Open a terminal in the project root and run:

```
cd "c:\Users\partg\OneDrive\Desktop\Python Project\backend"
python "Python code.py"
```

This launches the FastAPI backend at `http://127.0.0.1:8000`.

2. Start the frontend static server

Open a separate terminal in the project root and run:

```
cd "c:\Users\partg\OneDrive\Desktop\Python Project\frontend"
python -m http.server 5500
```

Then open in your browser:

```
http://127.0.0.1:5500/index.html
```

Figure 10 Github Repository Dashboard